Développement d'Applications Réparties

Chapitre 1 : Les sockets (1)

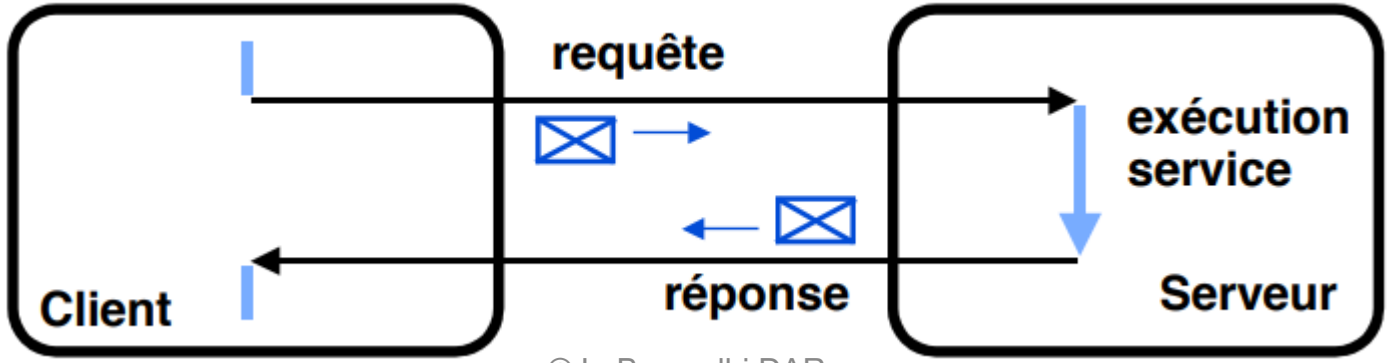
Séance 2 : 02/10/2026

Rappel : le modèle client/serveur

✓ **Le client** demande l'exécution d'un service

✓ **Le serveur** réalise le service

- Les deux sont généralement localisés sur deux machines distinctes (il s'agit parfois de la même machine)
- Communications par messages
  - **Requête** : paramètres d'appel, spécification du service requis
  - **Réponse** : résultats, indicateur éventuel d'exécution ou d'erreur
  - **Communication synchrone** (dans le modèle de base) : le client est bloqué en attente de la réponse



© L. Bayoudhi DAR

Exemple de client/serveur

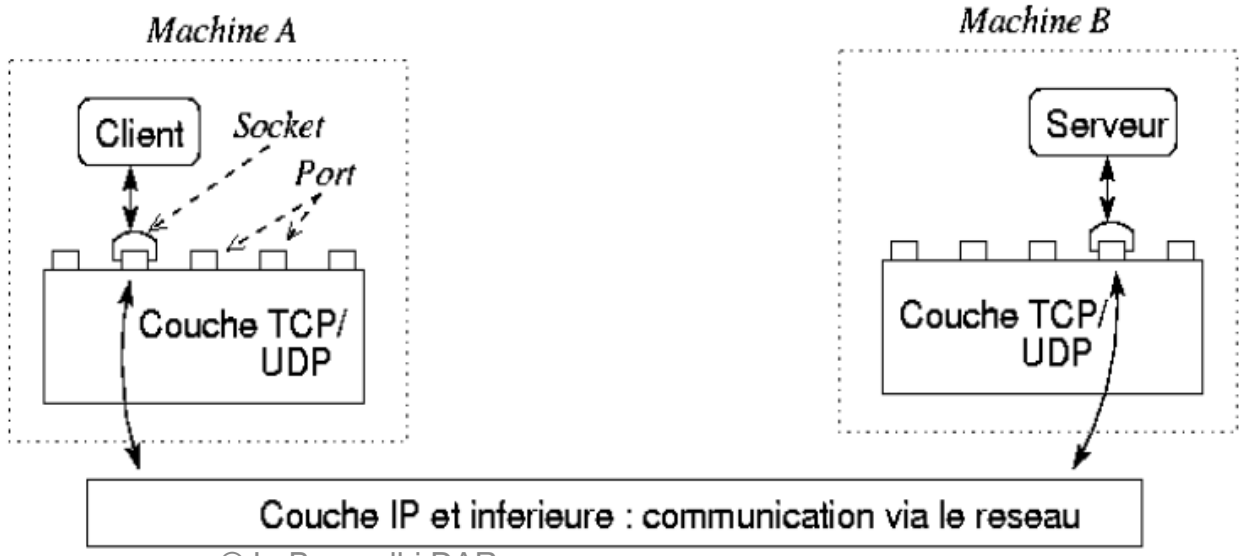
- Un serveur de fichiers
  - Des clients qui demandent des fichiers
- Comment gérer la concurrence ?
  - ✓ un processus serveur ⇔ un client servi
  - ✓ plusieurs clients ⇒ plusieurs processus serveur
- Quel protocole utiliser ? (**ensemble de règles** qui régissent les **échanges de données)**
  - ✓ le client envoie le nom du fichier
  - ✓ le serveur renvoie la taille puis les données
- comment gère-t-on les erreurs ?
  - ✓ Le fichier demandé n'existe pas → le serveur doit renvoyer un message d'erreur (comme "404 Not Found" en HTTP).
  - ✓ Problème de droit d'accès → renvoyer un code d'erreur.
  - ✓ Problème réseau → prévoir une reconnexion ou un message clair.

Rappel : Mise en œuvre du modèle client/serveur

- Besoin d'un support pour transporter les informations entre le client et le serveur (C/S)
  - ► Bas niveau
    - Utilisation directe du transport : sockets (construits sur TCP ou UDP) (détails dans ce chapitre)
  - ► Haut niveau
    - Intégration dans le langage de programmation : RPC ou Remote Procedure Call (construits sur sockets) (chapitre 2 et les autres)
- Nécessité d'établir un protocole entre le client et le serveur pour qu'ils se comprennent

Les sockets

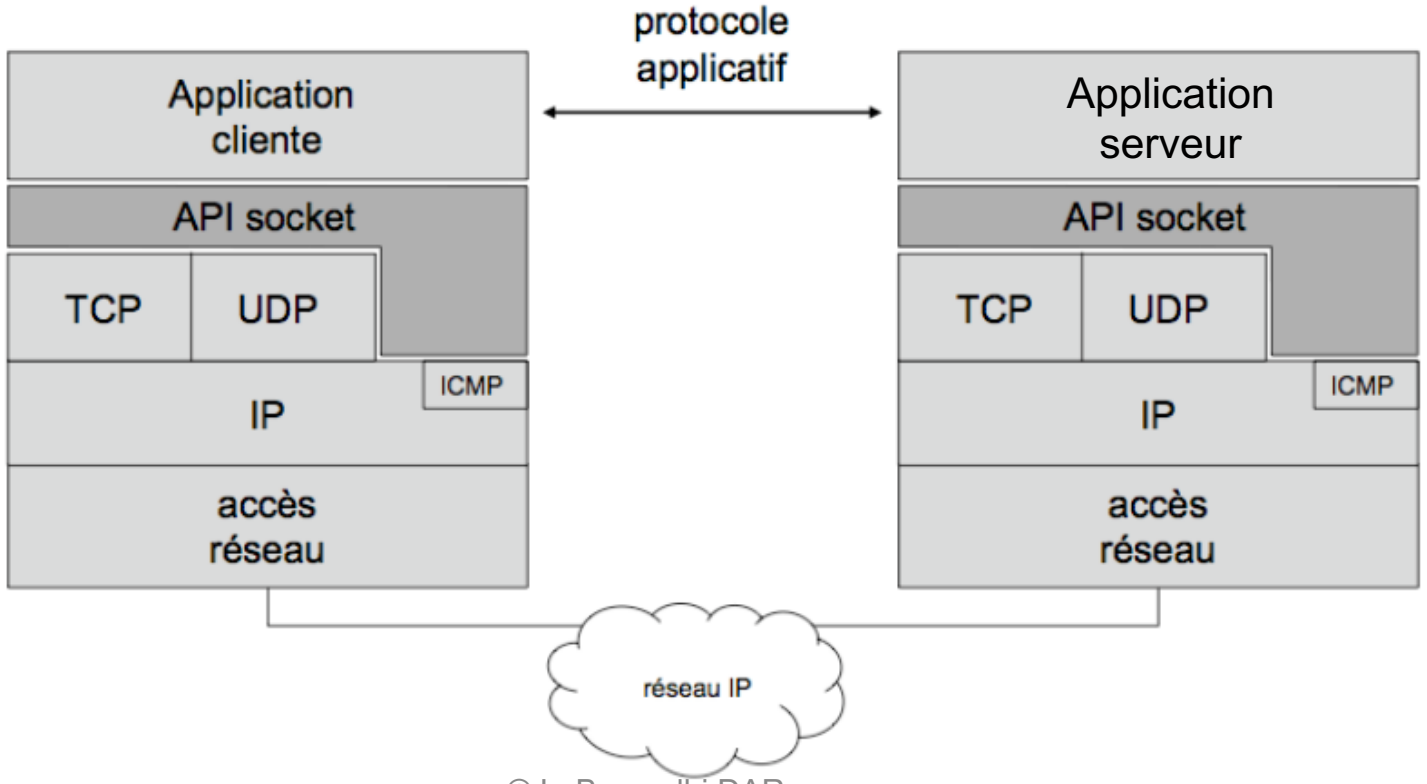
- Le terme socket signifie douille, prise électrique femelle, ce sur quoi on branche quelque chose
- La socket est un mécanisme de communication permettant d'utiliser l'interface de transport (TCP-UDP)
- L'API (Application Programming Interface) Socket
  - ✓ permet aux programmes d'échanger des données
  - ✓ les applications client/serveur ne voient les couches de communication qu'à travers l'API socket (abstraction)
  - ✓ Une socket est liée localement à un port. Elle permet la communication avec un port distant sur une machine distante : c'est-à-dire avec une application distante



© L. Bayoudhi DAR

L'API Socket

- Les **sockets** sont une **interface de programmation**
  - ✓ **Ils** permettent de se connecter, d'envoyer et de recevoir des données.
  - ✓ Le mode de transport des données dépend du protocole sous-jacent utilisé par le socket :
    - •**Socket TCP** : la communication utilisera les mécanismes de TCP
    - •**Socket UDP** : les données seront envoyées selon les règles de UDP.



Source Christine Bulfone : Le client/Serveu©r et l'API socket

© L. Bayoudhi DAR

L'API Socket

- Mode connecté (protocole TCP)
  - ✓ Ouverture d'une liaison, suite d'échanges, fermeture de la liaison
  - ✓ Le serveur préserve son état entre deux requêtes
  - ✓ Garanties de TCP : ordre, contrôle de flux, fiabilité
  - ✓ Adapté aux échanges ayant une certaine durée (plusieurs messages)
  - ✓ Lenteur des transmissions
  - ✓ Applications : email, envoi d'emails, transfert de fichiers.
  - ✓ Vu comme un «service téléphonique»





L'API Socket



- Mode non connecté (protocole UDP)
  - ✓ Les requêtes successives sont indépendantes
  - ✓ Pas de préservation de l'état entre les requêtes
  - ✓ Le client doit indiquer son adresse à chaque requête (pas de liaison permanente)
  - ✓ Pas de garanties particulières (UDP)
  - ✓ Arrivée dans le bon ordre non garantie
  - ✓ Non fiabilité des transmissions
  - ✓ Adapté aux échanges brefs (réponse en 1 message)
  - ✓ Rapidité des transmissions
  - ✓ Applications audio/vidéo, streaming vidéo, jeux en ligne
  - ✓ Vu comme un «service postal»



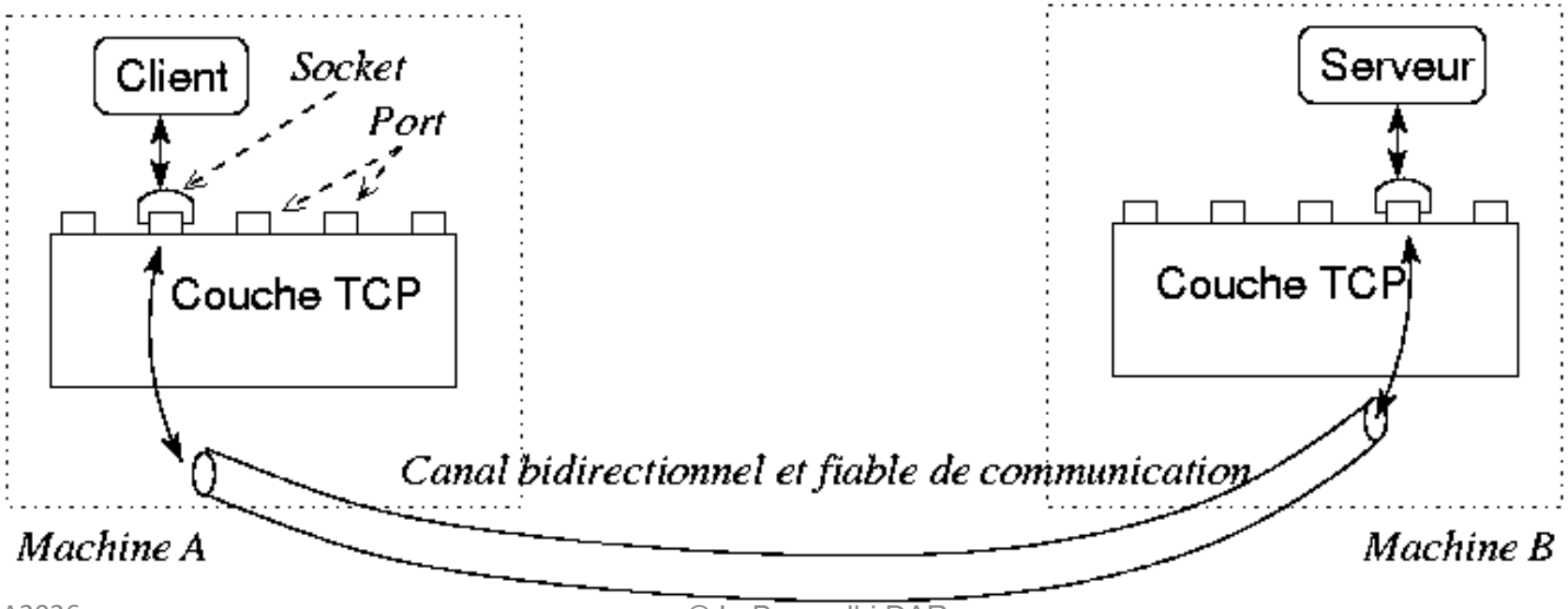
L'API Socket

- Points communs 
  - ✓ Le client a l'initiative de la communication : le serveur doit être à l'écoute
  - ✓ Le client doit connaître la référence du serveur [adresse IP, n° de port]
    - il peut la trouver dans un annuaire si le serveur l'y a enregistrée au préalable,
    - ou la connaître par convention : n°s de port pré-affectés
  - ✓ Le serveur peut servir plusieurs clients (1 thread unique ou 1 thread par client)

© L. Bayoudhi DAR

Sockets TCP : En mode connecté

- Données envoyées dans un « tuyau » et non pas par paquet : Flux de données
- Une socket est un des deux bouts du tuyau
- Chaque application a une socket locale pour gérer la communication à distance



© L. Bayoudhi DAR

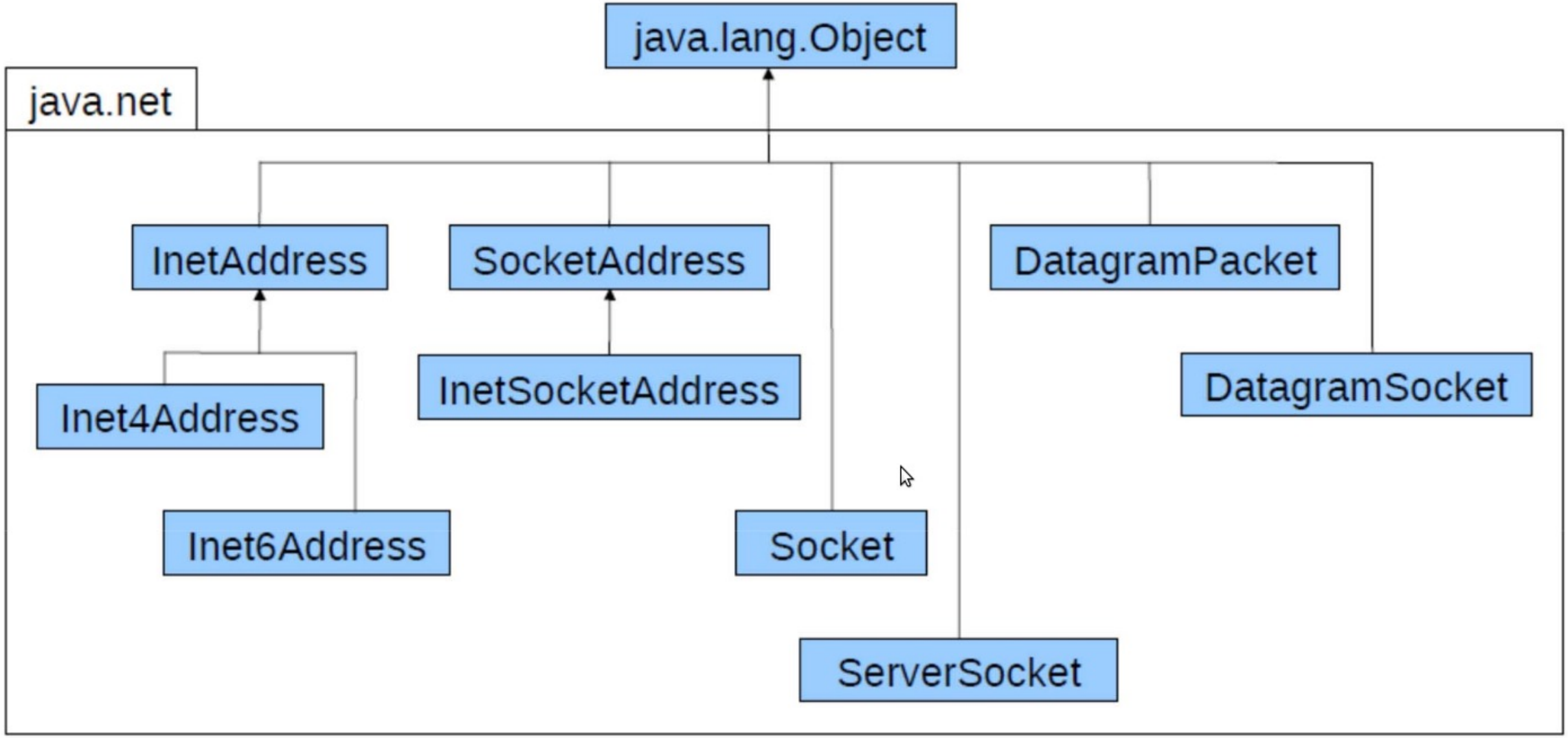
Sockets TCP : principe

Client

1. Crée une socket
2. Se connecte au serveur en donnant l'adresse socket distante (adresse IP du serveur et numéro de port du service).
3. Le système d'exploitation attribue pour cette connexion automatiquement un numéro de port local au client
4. Lit et/ou écrit (envoi et réception des messages) sur la socket
5. Ferme la socket

Serveur

1. Crée une socket
2. Le serveur lie une socket d'écoute sur un certain port bien précis
3. Se met à l'écoute des connexions entrantes
4. Pour chaque connexion entrante
    i. Accepte la connexion (un nouveau socket de service créé, ce qui permet de lancer un thread ou un processus fils pour gérer cette connexion et dialoguer avec le client)
    ii. Lit et/ou écrit sur la nouvelle socket
5. Ferme la socket créé


java.lang.Object
java.net
InetAddress
SocketAddress
DatagramPacket
Inet4Address
InetSocketAddress
DatagramSocket
Inet6Address
Socket
ServerSocket

Les sockets TCP en Java

- **Classes du package java.net utilisées pour communication via TCP**
  - ►**InetAddress :** codage des adresses IP
  - ►**Socket** : socket mode connecté
  - ►**ServerSocket** : socket d'attente de connexion du coté serveur

© L. Bayoudhi DAR

Les sockets TCP en Java

- **Classes du package java.net utilisées pour communication via TCP**

  ►**InetAddress** : codage des adresses IP

  - **Constructeurs :** Pas de constructeurs, on passe par des méthodes statiques pour créer un objet
  - **Méthodes** :
    - ❑ public static InetAddress getByName(String host) throws UnknownHostException : Crée un objet InetAddress identifiant une machine dont le nom est passé en paramètre
      - ➢ L'exception est levée si le service de nom (DNS...) du système ne trouve pas de machine du nom passé en paramètre sur le réseau.
    - ❑ public static InetAddress getLocalHost() throws UnknownHostException : Retourne l'adresse IP de la machine sur laquelle tourne le programme, c'est-à-dire l'adresse IP locale.
    - ❑ public String getHostName() : Retourne le nom de la machine dont l'adresse est codée par l'objet InetAddress.

Les sockets TCP en Java

- **Classes du package java.net utilisées pour communication via TCP**
  - ►**Socket** : socket mode connecté
  - **Constructeurs :**
    - ❑ public Socket(InetAddress address, int port) throws IOException : crée une socket locale et la connecte à un port distant d'une machine distante identifiée par le couple address/port.
    - ❑ public Socket(String name, int port) throws IOException,UnknownHostException : idem mais avec nom de la machine au lieu de son adresse IP codée.
      - — lève l'exception UnknownHostException si le service de nom ne parvient pas à identifier la machine.

Les sockets TCP en Java

- **Classes du package java.net utilisées pour communication via TCP**

  ►**Socket** : socket mode connecté

  - **Méthodes**

    ✓Méthodes « get »

    ❑ int getPort() : renvoie le port distant avec lequel est connecté la socket.

    ❑ InetAddress getAddress() : renvoie l'adresse IP de la machine distante.

    ❑ int getLocalPort() : renvoie le port local sur lequel est liée la socket.

Les sockets TCP en Java

- **Classes du package java.net utilisées pour communication via TCP**

  ►**Socket** : socket mode connecté

  - **Méthodes**

    ✓Emission/réception de données : on récupère les flux d'entrée/sorties associés à la socket à travers :

    ❑ OutputStream getOutputStream() : retourne le flux de sortie permettant d'envoyer des données via la socket.

    ❑ InputStream getInputStream() : retourne le flux d'entrée permettant de recevoir des données via la socket.

    ✓Fermeture d'une socket :

    ❑ public close() qui sert à fermer la socket et rompt la connexion avec la machine distante.

Les sockets TCP en Java

- **Classes du package java.net utilisées pour la communication via TCP**

  ►**ServerSocket** : Socket d'attente de connexion, coté serveur uniquement

  - **Constructeurs :**

    ❑ public ServerSocket(int port) throws IOException : Crée une socket d'écoute (d'attente de connexion de la part de clients). La socket est liée au port dont le numéro est passé en paramètre. L'exception est levée notamment si ce port est déjà lié à une socket.

  - **Méthodes :**

    ❑ Socket accept() throws IOException : Attente de connexion d'un client distant. Quand connexion est faite, retourne une socket permettant de communiquer avec le client : socket de service.

Sockets TCP : principe d'une application Client/Serveur

Création d'un Serveur

```
public static void main(String[]  args){
// La première étape : création de
l'objet ServerSocket

ServerSocket ss = new ServerSocket(2026);

// La deuxième étape : création de
l'objet Socket côté
serveur suite à la connexion du client
Socket s = ss.accept();

System.out.println("un client est connecté");

// échange des données



// La dernière étape : fermeture
de la socket
 s.close();}
```

Création d'un client

```
public static void main(String[]  args){


// La première étape : demande de
connexion création de l'objet Socket
côté client

Socket s = new Socket("localhost",2026);

System.out.println("je suis un client connecté")

// échange des données



// La dernière étape : fermeture
de la socket
   s.close(); }
```


© L. Bayoudhi DAR

Sockets TCP : principe d'une application Client/Serveur

Création d'un Serveur

```
public static void main(String[] args){
// La première étape : création de l'objet ServerSocket
ServerSocket ss = new ServerSocket(2026);
// La deuxième étape : création de l'objet Socket côté serveur suite à la connexion du client
Socket s = ss.accept();
System.out.println("un client est connecté");
// échange des données
InputStream is = s.getInputStream();
int nb =is.read();
int res = nb*2;
OutputStream os = s.getOutputStream();
os.write(res);
// La dernière étape : fermeture de la socket
s.close();}
```

Création d'un client

```
public static void main(String[] args){
// La première étape : demande de connexion création de l'objet Socket côté client
Socket s = new Socket("localhost",2026);
System.out.println("je suis un client connecté");
// échange des données
OutputStream os = s.getOutputStream();
os.write(27);
InputStream is = s.getInputStream();
int res = is.read();
System.out.println("le résultat reçu est "+res);
// La dernière étape : fermeture de la socket
s.close(); }
```